There are different strategies for organizing Kubernetes manifests. For the Catalog Service application, create a “k8s” folder in the project root (catalog-service). We will use it to store the manifests for the application.

NOTE If you haven’t followed along with the examples implemented in the previous chapters, you can refer to the repository accompanying the book (https://github.com/ThomasVitale/cloud-native-spring-in-action) and use the project in Chapter07/07-begin/catalog-service as a starting point.

Let’s start by creating a deployment.yml file inside the catalog-service/k8s folder. As you saw in figure 7.5, the first sections you need to include are `apiVersion`, `kind`, and `metadata`.

Listing 7.1 Initializing a Deployment manifest for Catalog Service

```
apiVersion: apps/v1          ❶
kind: Deployment             ❷
metadata:
  name: catalog-service      ❸
  labels:                    ❹
    app: catalog-service     ❺
```

❶ The API version for Deployment objects

❷ The type of object to create

❸ The name of the Deployment

❹ A set of labels attached to the Deployment

❺ This Deployment is labeled with “app=catalog-service.”

NOTE The Kubernetes API can change over time. Make sure you always use the API supported by the version of Kubernetes you’re running. If you have followed along so far, you shouldn’t have this problem. But if it happens, kubectl will return a very descriptive error message telling you exactly what’s wrong and how to fix it. You can also use the `kubectl explain <object_ name>` command to check the API version supported by your Kubernetes installation for a given object.

The `spec` section of a Deployment manifest contains a `selector` part to define a strategy for identifying which objects should be scaled by a